THE GENERATIVIZATION SERIES · BOOK I

# Agentic Loops

*Building Software Where Models Decide What Happens Next*

## Chapter 2: The Ghost in the Loop

This is a free sample chapter from Agentic Loops, by José Luis Martin Matias, typeset exactly as it appears in the book. The lab the chapter ends with, labs/ch02_the_ghost_in_the_loop_lab.py, is in the same repository and runs without an API key or any third-party package.

https://github.com/jmartinmatias/agentic-loops

© 2026 José Luis Martin Matias. All rights reserved. The code listings are released under the MIT License.

CHAPTER 2

# The Ghost in the Loop

*Putting a model inside*

> *The question is not whether machines think, but whether people can stop themselves from believing they do.*
>
> Claude, a language model, 2026

In 1966, at MIT, a computer scientist named Joseph Weizenbaum built a program to make a point about how shallow machine conversation really was. It took him a few hundred lines of pattern matching. He called it ELIZA, after Eliza Doolittle, the flower girl who could be taught to speak like a duchess, and he gave it a script named DOCTOR that imitated a particular school of psychotherapist: the kind that mostly turns your own words back on you. Tell it *my boyfriend made me come here* and it might reply *your boyfriend made you come here?* Tell it you are depressed and it would ask you to say more about that. There was no understanding anywhere in the machine. Keyword, rank, decompose, reassemble, print. A parlor trick, built to expose parlor tricks.

Then his secretary sat down at the terminal. She had watched him build the thing. She knew, in the way you know about a colleague's project, exactly what it was. After a few exchanges she asked Weizenbaum to leave the room, because the conversation had become private.

He never fully recovered from that. People who knew precisely what ELIZA was confided in it anyway, and when he proposed reading the transcripts, he was accused of spying on them. Weizenbaum spent much of the rest of his career, including an entire anguished book, warning the world about what he had accidentally demonstrated: that humans will extend trust, intimacy, and the presumption of a mind to anything that holds up its end of a conversation. The phenomenon still bears his program's name. Researchers call it the ELIZA effect.

Here is the detail that matters for us, the one that makes ELIZA the perfect front door to this chapter rather than just a good story. ELIZA was *frozen*. Completely, provably,

line-by-line frozen. Every response it ever gave was the deterministic output of rules a person had written; you could take any exchange, however uncanny, and trace it back to a numbered pattern in the DOCTOR script. In the vocabulary you earned in Chapter 1: the body was an artifact, the flow was an artifact, and the ghost was not in the machine at all. The ghost was in the reader.

Sixty years later the situation has inverted, and the inversion is this book's subject. The systems we are about to build really do generate their behavior at runtime. The transition function is no longer a page of rules anyone wrote; it is the output of a model, produced fresh each step, for states no author enumerated. The ghost has, in a meaningful engineering sense, moved into the machine. And so our problem is the mirror image of Weizenbaum's. His nightmare was people trusting a frozen script because it sounded alive. Ours is building the machinery that lets us justifiably trust something genuinely fluid, for reasons we can name, log, and defend.

In this chapter we perform the melt. We take the loop you built in Chapter 1, replace one line, and then spend the rest of the chapter honestly confronting what that one line costs: a new species of error, a new job for every guarantee, and a new definition of the word "working."

## 2.1 One Line of Difference

Put Chapter 1's loop core side by side with its melted successor:

```
# Chapter 1                              # Chapter 2
body = flow[state.phase]                 decision = model(render(state))
nxt = body(state)                        nxt = apply(decision, state)
```

On the left, a table you wrote decides what happens next, and a function you wrote does it. On the right, the state is rendered into a context, a model reads that context and *emits a decision*, and the shell applies it. The loop skeleton is identical: same while, same state, same termination checks standing guard. What changed is the answer to one question, the question this whole book orbits: *who decides what happens next?* Yesterday, you did, at your desk, in advance, for every case you could think of. Now the running system does, at runtime, for the case actually in front of it.

Notice what did *not* melt, because the restraint is the engineering. The state model: yours. The inventory of available actions: yours. The rendering of state into context: yours. The parsing and validation of what comes back: yours. The termination authority: emphatically yours. We have taken the classical loop and hollowed out

exactly one chamber, the decision, and filled it with inference. Everything surrounding that chamber is as frozen as it was on Eagle.

### Wait. Is choosing the next action not control flow?

A sharp reader will object that deciding what happens next sounds like flow, which this book promised to keep frozen. The distinction is worth stating precisely, because two entire sequels live on the far side of it. The *flow* here is the authored skeleton: one decision per iteration, a fixed vocabulary of actions, a fixed termination authority, a shell whose structure never changes shape at runtime. The model fills a slot *inside* each step. It cannot add a branch to the program, spawn a second loop, restructure the computation, or overrule the watchdog. When the skeleton itself becomes something the system generates, that is a different transformation with different dangers, and it gets its own book. For now: the model decides *within* the step. The arrangement *of* steps stays yours.

## 2.2 A Policy, Not an Oracle

You have almost certainly called a model before: paste in a document, get a summary, done. That is a model used as an *oracle*: a stateless function, consulted once, whose mistakes are annoying but contained. The moment you place the same model inside a loop, it becomes something categorically different: a *policy*, a standing rule for choosing actions, whose outputs become the inputs of its own future.

That feedback arrow changes everything, in both directions at once.

Downward first. Errors compound. An oracle's wrong answer sits inertly on the screen; a policy's wrong action changes the state, which changes the next context, which conditions the next decision. One bad step and the model is now navigating a world that its own mistake created, drifting further from anything its training ever resembled. Left unattended, a looped model does not make one error; it makes an error and then builds on it, with the fluency of something that has never once experienced doubt.

But the same arrow points upward. Feedback is also how the system gets to *notice*. An oracle can never discover it was wrong; there is no afterward. A policy acts, observes the consequence, and gets a fresh chance to respond to reality rather than to its expectation of reality. Every self-correcting behavior you have seen an agent perform, every retried command and revised approach, is this arrow doing its good work. The loop is what turns a text generator into a participant: something that can be wrong *and then find out*.

Two sober footnotes before we move on. First, whatever multiplies also multiplies costs: a policy consulted thirty times per task incurs thirty times the latency and thirty times the bill of an oracle, a fact with an entire chapter of consequences waiting at the end of this book. Second, and write this somewhere visible: the *system* is not the model. The system is the model plus the shell plus the accumulated state, and it is the system, never the raw model, that your users experience and your auditors examine. Most of what the industry calls model failures are, on inspection, system design failures wearing the model as a mask.

## 2.3 Two Species of Wrong

Chapter 1's machine could fail, and did, in front of half a billion people. But recall *how* it failed: loudly, legibly, and by the book. The 1202 was a machine announcing its own distress with a numbered code that a person had assigned to that exact condition in advance. Classical failures are like that as a family. They are deterministic, so the same input reproduces them on demand. They are diagnosable, because a stack trace points at authored lines. And they are *fixable at the source*: find the wrong line, change it, and that failure is gone from the universe.

Model errors are a different species, and the difference is the intellectual adjustment this chapter is really asking of you. A model's error arrives fluent, well-formatted, and confident, indistinguishable in tone from its correct output. It may not reproduce: sample again and the error is gone, or different. It shifts with context in ways no line of code explains, because there is no line of code; there are weights, and you do not get to edit them. A program that is wrong crashes or contradicts itself. A model that is wrong *testifies*. Programs fail like machines. Models fail like witnesses: sincerely, plausibly, and sometimes about things that never happened.

The practical taxonomy has four entries, and you will meet three of them, in the flesh, in this chapter's lab:

**Malformed output.** You asked for JSON; you received an enthusiastic paragraph. The cheapest error, because it is mechanically detectable at the boundary.

**Out-of-space output.** Well-formed, and requesting an action that does not exist. The model has invented a capability, which sounds exotic until you watch it happen on a Tuesday.

**Plausible-but-wrong output.** Well-formed, in-vocabulary, and a bad idea: the wrong action for this state, chosen with perfect syntax. The most expensive entry, because no parser can catch it; only a verifier that knows what *good* means can.

**Premature success.** The model declares the task complete while the work sits unfinished. A special case of the previous entry, so common and so corrosive that it earns its own line and, in the lab, its own guardrail.

Now the adjustment itself, stated as bluntly as it deserves. With classical errors, your posture is *debugging*: hunt the cause, edit the source, eliminate the failure. With model errors, that posture is unavailable. There is no line to fix. Your posture becomes *bounding*: assume every species above will occur at some frequency forever, and build the surrounding machinery so that when they occur, they are detected, contained, and recovered from, with the incident on the record. You do not fix a witness. You cross-examine one. Which is why the rest of this chapter is about the courtroom.

## 2.4 How to Make a Poet Fill In a Form

A language model wants to write prose. That is not a flaw; it is the substance of the thing. Ask one what to do next and it will, by default, answer the way a thoughtful colleague answers: in sentences, with hedges, context, and the occasional flourish. Charming in a colleague. Useless to a shell that must now decide which function to call.

The amateur response is to accept the prose and go fishing in it with regular expressions. Do not do this. You will spend your career maintaining a parser for the world's most creative format, and you will still lose, because the format changes whenever the weather does.

The professional response is to shrink the doorway. You cannot stop the poet writing poetry, but you can hand the poet a form on which poetry has no field:

```
{"action": "compute", "reason": "accounts loaded, need overall churn"}
```

One object. One `action`, drawn from an enumerated vocabulary you published in the prompt. One short `reason`. That is the entire interface between the fluid filling and the frozen shell, and every property of it is doing deliberate work. The enumeration converts an open-ended generation problem into a selection problem, which models are markedly better at and verifiers can check mechanically: the reply either names a real action or it does not, a decidable question, answered in microseconds, with no judgment involved. And the `reason` field is not decoration. It is *decision provenance*: a contemporaneous note in the model's own words for why this step happened, written into the trail beside the action it explains. Six chapters from now,

when a trajectory goes somewhere strange, those little clauses will be the difference between an investigation and a shrug.

Modern APIs will meet you more than halfway here, with function-calling interfaces and schema-constrained decoding that make malformed output rare rather than routine. Use them. But treat them as reducing the frequency of boundary violations, never as eliminating the boundary. The shell validates everything that crosses, however it was produced, because the shell's authority cannot depend on the good manners of the thing it exists to bound. And when validation fails? You do the one thing you could never do to a crashed program: you hand the form back and say, politely and mechanically, *this was not valid, try again*. A retry against a fresh sample is a genuinely new draw, not a rerun of a deterministic bug. It is the first entry in a long catalog of repair moves that exist only on this side of the melt.

## 2.5 The Shell Stays Frozen

Assemble the pieces and a shape emerges, one you will build in every chapter from here to the end of the book. It is a sandwich, and only the middle layer is new:

```
frozen    render(state)  -> context          you wrote this
fluid         model(context) -> text         generated, each step
frozen    parse -> validate -> apply -> log  you wrote this too
```

Everything above and below the model call is classical software, held to Chapter 1's standards without apology. The top slice decides what the model gets to see: which facts of the state, in what form, with what instructions and what vocabulary of actions. The bottom slice decides what the model gets to *do*: parse the reply, check it against schema and vocabulary and postconditions, apply it to the state through functions you wrote, and record the whole exchange in the trail. In between sits the one component you did not author and cannot inspect, held on both sides by components you did and can.

Read the sandwich as a statement about responsibility and it says something almost legal: *the model proposes; the shell disposes.* Nothing becomes real in your system because a model said it. It becomes real because the shell, applying rules written in advance by an accountable person, accepted it. That sentence is the entire trust architecture of this book in miniature, and it is why the shell must stay boring. Every clever behavior you are tempted to add to the shell is a behavior you can no longer rely on when the filling misbehaves.

One more Chapter 1 echo, because it is load-bearing: the trail. In a deterministic loop, logging was good hygiene. Here it is closer to a constitutional requirement. The model's decision process is invisible; the trail is the only place the system's history exists in inspectable form. Every prompt rendered, every reply received, every rejection and retry and reason: written down, replayable, and beyond the model's reach. In Chapter 9 this sandwich grows up and acquires a proper name, the harness, and a production-grade feature list. The shape will not change. The shape is the point.

## 2.6 G(body), Named

Time to say formally what we have done, and to update the picture we drew on Eagle.

```
G(body):  artifact(body)  ->  emission(body)
```

The step decision, which for the entire history of software has been an artifact (authored, reviewed, versioned, frozen), is now an emission: generated at runtime, per state, by a model. Redraw the stack:

```
 layer      | Meridian, Chapter 2              | authored when?
------------+----------------------------------+----------------
 GUARANTEE  | schema, vocabulary, watchdog,    |  design time
            | postconditions, repair budget    |   (LOCKED)
 FLOW       | the shell's skeleton             |  design time
 ACTOR      | one loop, one model, one role    |  design time
 BODY       | the decision, each step          |  RUNTIME (FLUID)
```

One row melted, three still frozen, and look at what happened to the guarantee row: it got *longer*. That is Chapter 1's inverse law arriving on schedule. The determinism we melted out of the body did not vanish; it moved into the locked layer, where it now lives as schemas, vocabularies, postconditions, and budgets, verified at runtime because it can no longer be assumed at design time.

There is a second migration in the diagram, quieter and easy to miss. In Chapter 1, the intelligence lived in the bodies (four functions that knew things) and the control was a dumb table. Now the intelligence lives in the decision, and the four functions have been demoted to something new: a vocabulary of capabilities, dumb, reliable, and waiting to be chosen. Hold that thought. In Chapter 7 those demoted functions acquire their proper name, tools, and turn out to be one of the most consequential design surfaces in the entire field.

Which leaves the question every melt must answer: *should* you? `G` is an operator, not an obligation, and applying it tastefully is a skill this book intends to build in you.

The rubric, in first draft: **melt where enumerating the cases is harder than verifying the outcome; keep frozen whatever is cheap to compute and expensive to get wrong.** Meridian's lab is a working demonstration of both halves. The decision melted, because scripting every path through an open-ended analysis is exactly the enumeration problem models dissolve. The arithmetic did not melt, and never will: churn is a division, division is free, and a model computing percentages is a liability you volunteered for. We asked the model to decide *that* computing churn is what happens next. We did not ask it to compute churn. Most of the bad agentic software in the world sits on the wrong side of that italicized line.

## 2.7 Lab: Melting the Meridian Loop

Same company, same ten accounts, same report. One difference, and by now you can name it in the series' own vocabulary: the FLOW table is gone. Nothing in this file knows the order of operations. The order will be an emission.

The model in the listing is a scripted stand-in, so the lab runs anywhere, deterministically, with no API key, and so we can choreograph its failures on cue. Everything on the shell side is exactly what you would ship.

```
"""Meridian, melted: Chapter 1's loop with a model deciding the next step.

Book I, Chapter 2 lab. The shell is frozen. The filling is not.
"""

import json
from dataclasses import dataclass, replace

# ---------------------------------------------------------------
# STATE and ACTIONS -- carried over from Chapter 1, minus the FLOW
# table. The arithmetic stays frozen. Only the *decision* melts.
# ---------------------------------------------------------------

RAW = (
    {"id": "A-01", "plan": "basic",      "cancelled": True},
    {"id": "A-02", "plan": "basic",      "cancelled": True},
    {"id": "A-03", "plan": "basic",      "cancelled": False},
    {"id": "A-04", "plan": "pro",        "cancelled": True},
    {"id": "A-05", "plan": "pro",        "cancelled": False},
    {"id": "A-06", "plan": "pro",        "cancelled": False},
    {"id": "A-07", "plan": "pro",        "cancelled": False},
    {"id": "A-08", "plan": "enterprise", "cancelled": False},
    {"id": "A-09", "plan": "enterprise", "cancelled": False},
    {"id": "A-10", "plan": "enterprise", "cancelled": False},
)
```

```
@dataclass(frozen=True)
class State:
    step: int = 0
    accounts: tuple = ()
    churn_rate: float | None = None
    segments: tuple = ()
    report: str | None = None

def load(s: State) -> State:
    return replace(s, accounts=RAW)

def compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, churn_rate=cancelled / len(s.accounts))

def segment(s: State) -> State:
    plans = sorted({a["plan"] for a in s.accounts})
    segs = tuple(
        (p,
         sum(1 for a in s.accounts if a["plan"] == p and a["cancelled"]) /
         sum(1 for a in s.accounts if a["plan"] == p))
        for p in plans
    )
    return replace(s, segments=segs)

def write_report(s: State) -> State:
    lines = [f"MERIDIAN CHURN REPORT  |  overall: {s.churn_rate:.0%}"]
    lines += [f"  {plan:<12} {rate:>4.0%}" for plan, rate in s.segments]
    return replace(s, report="\n".join(lines))

ACTIONS = {"load": load, "compute": compute,
           "segment": segment, "write_report": write_report}

# ---------------------------------------------------------------
# RENDER -- the frozen half of the shell, on the way in.
# ---------------------------------------------------------------

def render(s: State) -> str:
    facts = [
        f"accounts loaded: {bool(s.accounts)}",
        f"churn computed: {s.churn_rate is not None}",
        f"segments computed: {bool(s.segments)}",
        f"report written: {s.report is not None}",
    ]
    return (
        "You are running Meridian's churn analysis, one step at a time.\n"
        "Current state:\n  " + "\n  ".join(facts) + "\n"
        f"Available actions: {', '.join(ACTIONS)}, or finish.\n"
        'Reply with JSON only: {"action": "<name>", "reason": "<short>"}'
    )

# ---------------------------------------------------------------
# MODEL -- the fluid filling. Scripted here; see the sidebar to go
```

```
# live. A real model receives render(state) and returns text.
# ------------------------------------------------------------------

class ScriptedModel:
    def __init__(self, replies):
        self.replies = list(replies)
    def __call__(self, prompt: str) -> str:
        return self.replies.pop(0)

# ------------------------------------------------------------------
# THE SHELL -- parse, validate, apply, log. Frozen, every line.
# ------------------------------------------------------------------

MAX_STEPS = 12     # progress budget: applied actions (the watchdog, as before)
MAX_REPAIRS = 4    # patience budget: tolerated bad replies before we abort

def run(model, state: State = State()):
    trail, repairs = [], 0
    while True:
        if state.step >= MAX_STEPS:
            trail.append(f"step {state.step}: TERMINATED -- watchdog")
            break
        if repairs >= MAX_REPAIRS:
            trail.append(f"step {state.step}: TERMINATED -- repair budget
  exhausted")
            break

        raw = model(render(state))

        try:                                      # guardrail 1: well-formed
            decision = json.loads(raw)
            action = decision["action"]
        except (json.JSONDecodeError, KeyError, TypeError):
            repairs += 1
            trail.append(f"step {state.step}: GUARDRAIL -- malformed reply "
                         f"rejected (repair {repairs}/{MAX_REPAIRS})")
            continue

        if action == "finish":                    # guardrail 3: postcondition
            if state.report is None:
                repairs += 1
                trail.append(f"step {state.step}: GUARDRAIL -- premature finish
  "
                             f"rejected, no report exists "
                             f"(repair {repairs}/{MAX_REPAIRS})")
```

```
                continue
            trail.append(f"step {state.step}: finish -- TERMINATED -- success")
            break

        if action not in ACTIONS:               # guardrail 2: real action only
            repairs += 1
            trail.append(f"step {state.step}: GUARDRAIL -- unknown action "
                         f"{action!r} rejected (repair
  {repairs}/{MAX_REPAIRS})")
            continue

        state = replace(ACTIONS[action](state), step=state.step + 1)
        trail.append(f"step {state.step - 1}: {action} "
                     f"({decision.get('reason', '')})")
    return state, trail
```

Walk the shell before running anything, because its architecture is section 2.5 made executable. Two budgets stand at the top of the loop, and they are guarding different things: `MAX_STEPS` bounds *progress*, exactly as in Chapter 1, while `MAX_REPAIRS` bounds *patience*, a resource the deterministic loop never needed because a deterministic loop never talks back. Then three guardrails, one per detectable species from section 2.3: a parse boundary for malformed output, a vocabulary check for invented actions, and a postcondition on `finish` for premature success. Note the etiquette of rejection: a guardrail never crashes and never punishes. It logs, spends a repair, and goes around again, which means the very next `render(state)` shows the model an unchanged world, the mechanical equivalent of handing back the form.

Run 1, a well-behaved model:

```
--- RUN 1: a well-behaved model ---
step 0: load (no data in state yet)
step 1: compute (accounts loaded, need overall churn)
step 2: segment (break churn down by plan)
step 3: write_report (all numbers ready)
step 4: finish -- TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

The same report as Chapter 1, and the trail even reads the same, with one addition that should stop you for a second: the parentheses. Each step now carries its reason, in the decider's own words, at the moment of deciding. And remember what is absent from the file entirely: nobody told this program the order. Load before compute, compute before segment: that ordering was generated, step by step, from a rendered description of the state. You are looking at the first emission of the series.

But Run 1 is the demo. Run 2 is the chapter. Same shell, and a model having a worse day: one malformed reply, one hallucinated action, one premature declaration of victory, three of the four entries on parade:

```
--- RUN 2: the same shell, a worse day ---
step 0: GUARDRAIL -- malformed reply rejected (repair 1/4)
step 0: load (fine, JSON it is)
step 1: GUARDRAIL -- unknown action 'email_the_ceo' rejected (repair 2/4)
step 1: compute (back to the plan)
step 2: segment (by plan tier)
step 3: GUARDRAIL -- premature finish rejected, no report exists (repair 3/4)
step 3: write_report (apparently we were not done)
step 4: finish -- TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

Read the trail like a flight recorder, because that is what it is. A reply that was not JSON: rejected at the boundary, one repair spent. An action that does not exist, `email_the_ceo`, invented from thin air with a perfectly plausible reason attached: rejected by the vocabulary, second repair. A claim of success with no report in the state: rejected by the postcondition, third repair. And then, the finding: **the final report is byte-for-byte identical to Run 1's.** A misbehaving model, inside a sound shell, produced exactly the outcome a well-behaved one did, three incidents on the record, one repair still in the bank. In Chapter 1, the guarantee's whole job was to stop a broken body from running forever. The job has grown: the guarantees now steer a fallible decider back onto the road, mid-journey, without ever once understanding the road themselves. That is the trade this book asked you to consider, shown in fourteen lines of terminal output. The behavior became fluid. The outcome did not.

### Going live

Replace the scripted stand-in with a real model by swapping one object, using the official SDK. Nothing else in the file changes, which is the entire argument of section 2.5:

```
import anthropic
client = anthropic.Anthropic()   # reads ANTHROPIC_API_KEY from the environment

def live_model(prompt: str) -> str:
    msg = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=200,
        messages=[{"role": "user", "content": prompt}],
    )
    return msg.content[0].text

final, trail = run(live_model)
```

Run it a handful of times and study the trails. The orders will sometimes differ, the reasons will always differ, and every run should still end in the same verified report. If a guardrail fires, congratulations: you have caught a wild specimen of section 2.3, and your shell handled it while you watched. Current model names and API details live at docs.claude.com.

Two closing instructions, in the Chapter 1 tradition.

**Practical:** extend the shell, not the model. Feed rejection reasons back into the next prompt and measure whether repairs drop. Add a precondition guardrail (`compute` should be refusable when no accounts are loaded) and script a model that trips it. Then write the nastiest test in the book so far: a scripted model that answers `finish` with a plausible reason before any work is done, forever, and confirm which budget catches it and what the trail says afterward.

**And in pencil**, circle two regions. The `render` function, currently thirteen honest lines: Chapter 8 lives entirely inside it, and it will not stay thirteen lines. And the three guardrails, currently catching what a parser can see: Chapters 10 through 12 are the story of teaching that layer to catch what only a verifier can.

## Where We've Landed

One line melted, and the whole trust model reorganized around it. A model inside a loop is a policy, not an oracle: its outputs feed its future inputs, which is simultaneously how errors compound and how self-correction becomes possible, and it means the unit of engineering is the system (model, shell, state, trail), never the model

alone. Model errors are a second species of wrong: fluent, probabilistic, and unfixable at the source, so the working posture shifts from debugging to bounding, from eliminating failures to detecting, containing, and recovering from them on the record. The interface that makes bounding tractable is a shrunken doorway (schema, enumerated vocabulary, decision provenance), and the structure that enforces it is the sandwich: a frozen shell in which the model proposes and the shell disposes. Formally, we performed `G(body)`: the step decision became an emission while actor, flow, and guarantees stayed frozen, and, exactly as Chapter 1 predicted, the guarantee layer grew to absorb the determinism the body gave up. And the melt comes with a rubric, not a mandate: generate the decision where enumeration is harder than verification; keep frozen whatever is cheap to compute and expensive to get wrong.

## What's Next

The shell you just built has a dangerously innocent line in it: `render(state)` assumed the state was worth rendering. In 1999 two teams of excellent engineers flew a spacecraft into Mars because a number crossed a boundary without its units, a lesson about observation and contracts that cost a spacecraft worth well over a hundred million dollars and opens the next chapter. Before a model can decide well, it has to know what the world actually looks like. That turns out to be a discipline of its own.

*Weizenbaum's secretary trusted a frozen script because it sounded alive. Bales trusted a fluid situation because everything around it held still. Between those two kinds of trust sits the whole craft of this book: the secretary could not say why she believed; Bales could. Build shells that put you, permanently, on Bales's side of that line.*